SENTINEL

Project status and implementation report

CONTINUATION REPORT	8 OCTOBER 2026

Executive summary

Sentinel is a Python prototype for detecting and reviewing network anomalies. It turns JSONL event streams into time-window features, scores suspicious activity, records alerts and audit actions in SQLite, and exposes results through a command-line workflow and a Streamlit dashboard. Work continued from the project prompt file at Prompt 7 and completed the remaining Prompts 8–11.

The continuation added an operational dashboard, bounded streaming ingestion, broader resilience and workflow coverage, and a reproducible demo and documentation. The current repository suite reports **50 passing tests**. Determinism checks also produced matching console output and database dumps across repeated runs. These results support prototype functionality; they do not establish production detection effectiveness.

50	4	60 / 30 s	5000
tests passing	attack paths covered	window / hop	demo events

At a glance

- **Detection flow:** ingest, feature windows, model scoring, policy gate, alert persistence, dashboard review.
- **Recent scope:** Prompts 8–11 from *OPENCODE_PROMPTS.md*.
- **Primary caveat:** current risk scoring can elevate confidently predicted Normal traffic; the synthetic benchmark is a smoke check, not a field evaluation.

1. System overview

Sentinel processes timestamped network events from JSON Lines input. The pipeline builds rolling time-window summaries, applies trained classifiers and a deterministic risk calculation, and persists alerts for policy decisions and analyst review.

Stage	Role	Current behavior
Input	Read JSONL events	Streaming parser with a 1,048,576-character line cap, malformed/hostile-number handling, skip counters and event-ID fallback.
Features	Aggregate activity	Timestamp-ordered 60-second windows with a 30-second hop; caps active-window event count and history structures.
Models and risk	Classify and score	Isolation Forest and Random Forest components feed alert evidence and a 0–100 risk score.
Policy and audit	Gate and record	Typer CLI applies a configurable maximum-risk policy; SQLite stores alerts and review/audit actions.
Dashboard	Review alerts	Streamlit filters, KPIs, risk/category summaries, top source IPs, evidence details, review resolution and optional auto-refresh.

Streaming and bounded-work safeguards

- Input is processed in bounded feature batches of 256 and alerts are persisted incrementally.
- Out-of-order timestamps are skipped and logged; noise events are counted but excluded from detection windows.
- A maximum of 100,000 events is allowed in a feature window; source and peer histories are capped.

2. Prompt continuation delivered

Prompt	Delivered work
8 — Dashboard	Implemented a review-oriented Streamlit dashboard with alert filtering, aggregate views, evidence inspection, status changes and optional periodic refresh.
9 — Resilience	Hardened parsing and timestamp handling, bounded feature and history structures, incremental persistence, counters and resilience coverage for large inputs.
10 — Validation	Expanded tests across ingestion, features, policy, audit, models, resilience, attack-signature alert paths and dashboard rendering/review actions.
11 — Handoff	Added setup/run/model/demo documentation, a demo script, risk and evidence explanations, configuration notes, saved metric snapshot and explicit limitations.

3. Verification and observed results

Verification recorded during the continuation work focused on functionality, reproducibility and a runnable demo.

Check	Observed result	Interpretation
Automated suite	50 passed in 57.81 seconds	Covers core pipeline, resilience, attack-to-alert cases and dashboard behavior.
CLI input parity	File input and stdin produced byte-identical stdout/stderr for the same demo; both returned policy exit code 1 at max risk 70.	Supports consistent CLI handling across the two input modes for this fixture.
Policy threshold	With max risk 100, the demo returned exit code 0 and policy passed.	Confirms the configured gate changes the result as expected for this run.
Repeated runs	Separate temporary SQLite databases had identical SQL dumps and console output.	Supports deterministic behavior for the fixed demo and environment.
Demo workflow	Generator, training and detection completed in a clean temporary copy.	Basic end-to-end handoff is runnable using the documented Bash invocation.

Metric snapshot and limits

The saved synthetic evaluation snapshot reports precision, recall and F1 of 1.0 across classes. The class supports are highly uneven: Normal 14,722; PortScan 1; BruteForce 5; LateralMovement 2; DataExfiltration 1 (14,731 total). The README notes that overlapping windows from the same synthetic episodes can appear across random train/test splits, while attack supports are very small. Treat these metrics as a pipeline smoke check only; they are not evidence of real-world performance.

Known scoring limitation

The risk calculation uses maximum class probability, including the Normal class. Confident Normal predictions can therefore contribute substantially to risk and produce many high-risk alerts. A prior demo run showed 9,182 high-risk alerts including Normal predictions. Risk interpretation should remain cautious until the scoring logic and calibration are reviewed against representative validation data.

Other prototype limits

- The included traffic and metric snapshot are synthetic; there is no evidence here of validation against a representative live network.
- A reproducible demo proves the workflow executes, not that alerts are operationally useful or that the system is deployment-ready.
- The prototype’s models, thresholds and policy defaults need domain review, calibration and monitoring before operational use.

4. Running the project

The project README is the source of truth for dependencies and exact commands. The documented demo entry point is `bash scripts/demo.sh`. The end-to-end sequence generates sample traffic, trains models and runs detection. The dashboard can be launched with the documented Streamlit command after installing project dependencies.

Suggested review path

- Read the README for installation, configuration, CLI options, risk formula and evidence format.
- Run the automated suite in the project environment.
- Run the demo and inspect generated alerts, policy exit status and SQLite review records.
- Open the dashboard against the generated database and test filters, evidence and review actions.

Project handoff

Key deliverables are the Sentinel source tree, *OPENCODE_PROMPTS.md*, the README, *scripts/demo.sh*, dashboard code, test suite and *docs/metrics.json*. Prompts 8–11 were implemented as the continuation after the Prompt 7 boundary.

Assessment

Sentinel now has a coherent local prototype workflow with usable alert review, bounded processing behavior, automated coverage and a repeatable demo. The strongest next engineering priority is correcting and validating risk scoring, followed by evaluation on representative, separated data and performance characterization under realistic stream conditions.

Report prepared from the project files and verification results available in the workspace on 8 October 2026.